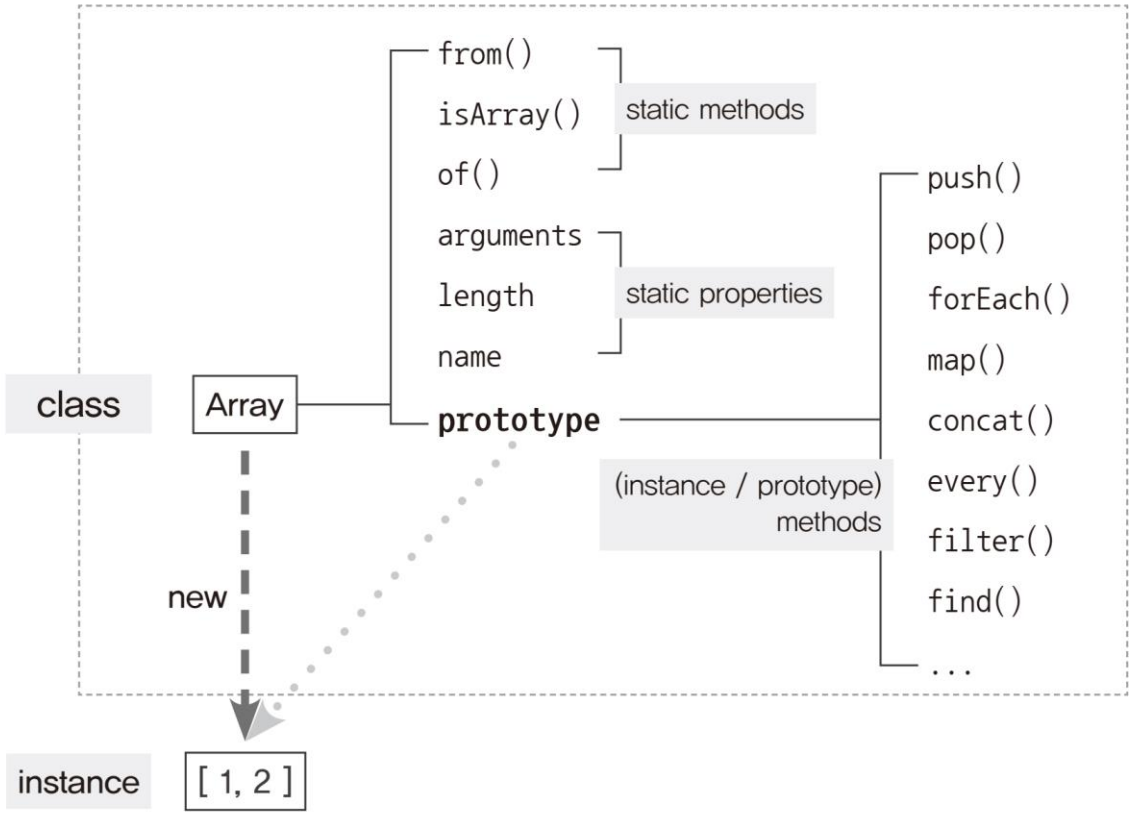
웹 시스템 개발 1

클래스

자바스크립트의 클래스

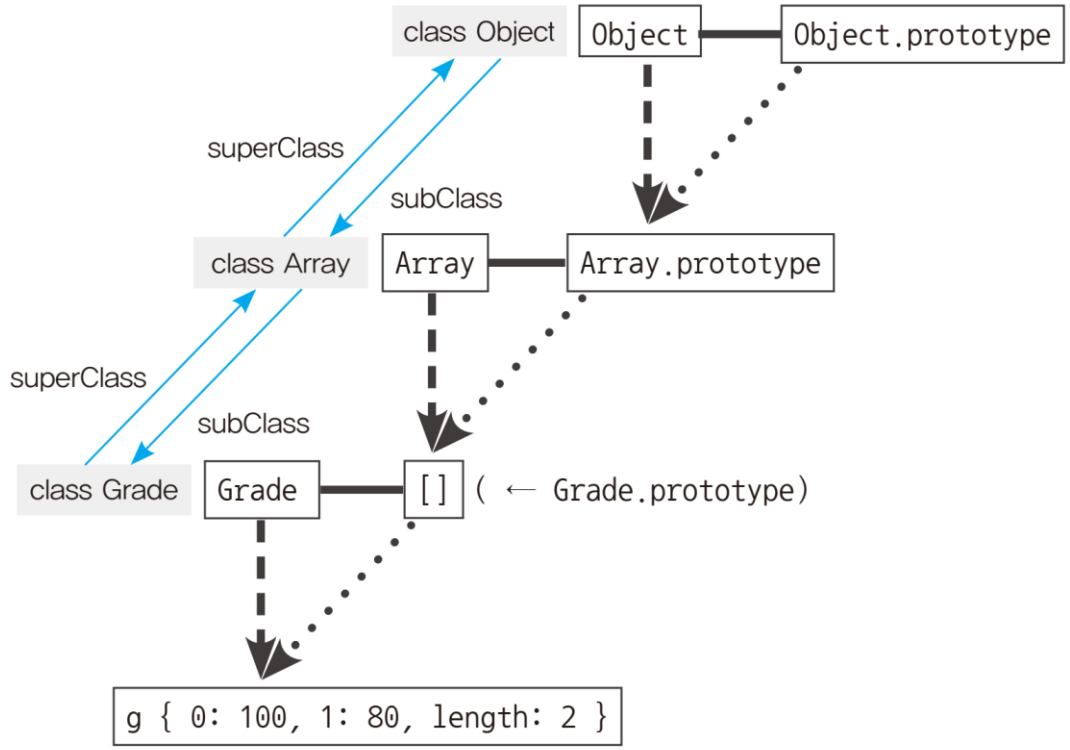
그림 7–4 프로토타입에 클래스 개념을 적용




```
1
2   var Rectangle = function (width, height) {
3       this.width = width;
4       this.height = height;
5   };
6
7   Rectangle.prototype.getArea = function () {
8       return this.width * this.height;
9   };
10
11  Rectangle.isRectangle = function (instance) {
12      return (
13          instance instanceof Rectangle
14      );
15  };
16
17  var rect = new Rectangle(3, 4);
18  console.log(rect.getArea());
19  console.log(Rectangle.isRectangle(rect));
20
```

클래스 상속과 프로토타입 체인 관계

그림 7-6 클래스 상속과 프로토타입 체인의 관계




```
1
2   var Grade = function () {
3       var args = Array.prototype.slice.call(arguments);
4       for (var i = 0; i < args.length; i++) {
5           this[i] = args[i];
6       }
7       this.length = args.length;
8   };
9
10  Grade.prototype = [];
11  var g = new Grade(100, 80);
12
13  console.log(g)
14  console.log(g[0], g[1])
15
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe ex.js
Array { '0': 100, '1': 80, length: 2 }
100 80
```

클래스 상속과 프로토타입 체인 관계

```
1
2   var Grade = function () {
3       var args = Array.prototype.slice.call(arguments);
4       for (var i = 0; i < args.length; i++) {
5           this[i] = args[i];
6       }
7       this.length = args.length;
8   };
9
10  Grade.prototype = [];
11  var g = new Grade(100, 80);
12
13  g.push(90);
14  console.log(g);
15
16  delete g.length;
17  g.push(70);
18  console.log(g);
19
```

```
OUTPUT  TERMINAL  DEBUG CONSOLE  PROBLEMS
C:\Program Files\nodejs\node.exe ex.js
Array { '0': 100, '1': 80, '2': 90, length: 3 }
Array { '0': 70, '1': 80, '2': 90, length: 1 }
```

- length 속성
  - 배열 인스턴스의 length 속성은 configurable 속성이 false 라서 삭제 불가능
  - Grade 클래스 인스턴스의 length 속성은 삭제 가능

- 17번 라인에서 push를 했을 때 0번째 인덱스에 70이 추가되고 length가 다시 1이 될 수 있었던 이유는?

클래스 상속과 프로토타입 체인 관계

```
1
2   var Grade = function () {
3       var args = Array.prototype.slice.call(arguments);
4       for (var i = 0; i < args.length; i++) {
5           this[i] = args[i];
6       }
7       this.length = args.length;
8   };
9
10  Grade.prototype = ['a', 'b', 'c', 'd'];
11  var g = new Grade(100, 80);
12
13  g.push(90);
14  console.log(g);
15
16  delete g.length;
17  g.push(70);
18  console.log(g);
19
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe ex.js
Array { '0': 100, '1': 80, '2': 90, length: 3 }
Array { '0': 100, '1': 80, '2': 90, '4': 70, length: 5 }
```

클래스 상속과 프로토타입 체인 관계

```
var Rectangle = function (width, height) {
    this.width = width;
    this.height = height;
};

Rectangle.prototype.getArea = function () {
    return this.width * this.height;
};

var rect = new Rectangle(3, 4);
console.log(rect.getArea());

var Square = function (width) {
    this.width = width;
};

Square.prototype.getArea = function () {
    return this.width * this.width;
};

var sq = new Square(5);
console.log(sq.getArea());
```

```
var Rectangle = function (width, height) {
    this.width = width;
    this.height = height;
};

Rectangle.prototype.getArea = function () {
    return this.width * this.height;
};

var rect = new Rectangle(3, 4);
console.log(rect.getArea());

var Square = function (width) {
    this.width = width;
    this.height = width;
};

Square.prototype.getArea = function () {
    return this.width * this.height;
};

var sq = new Square(5);
console.log(sq.getArea());
```

```
var Rectangle = function (width, height) {
    this.width = width;
    this.height = height;
};

Rectangle.prototype.getArea = function () {
    return this.width * this.height;
};

var rect = new Rectangle(3, 4);
console.log(rect.getArea());

var Square = function (width) {
    Rectangle.call(this, width, width);
};

Square.prototype = new Rectangle();

var sq = new Square(5);
console.log(sq.getArea());
```

그림 7-7 예제 7-6의 sq 인스턴스에 대한 콘솔 출력 결과

sq
Square's instance

sq.__proto__
Square.prototype
Rectangle's instance

sq.__proto__.__proto__
Rectangle.prototype
Object's instance

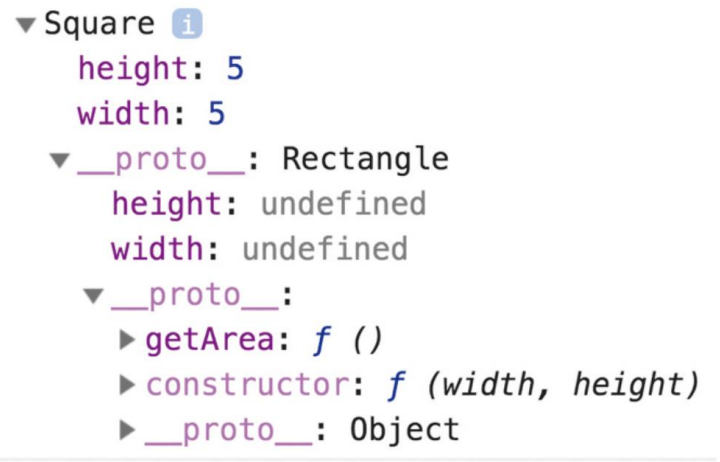
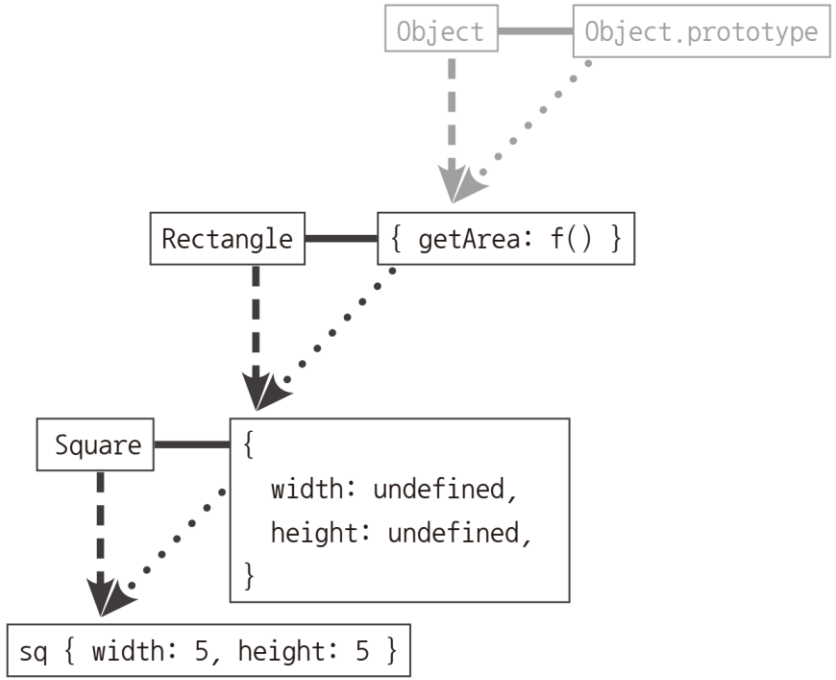


그림 7-8 Rectangle → Square 상속 관계 구현 (1) – 도식



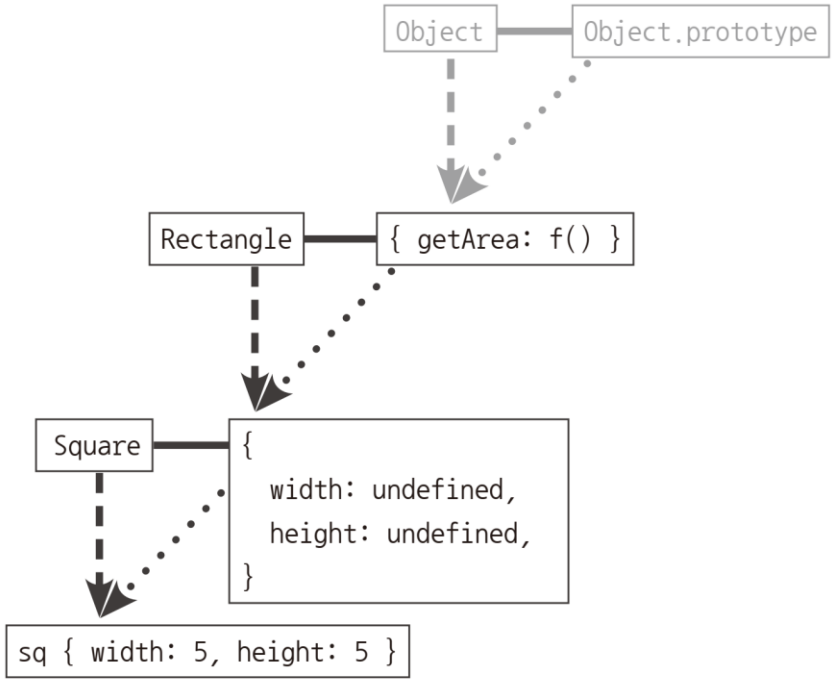

```
1
2   var Rectangle = function (width, height) {
3       this.width = width;
4       this.height = height;
5   };
6
7   Rectangle.prototype.getArea = function () {
8       return this.width * this.height;
9   };
10
11  var rect = new Rectangle(3, 4);
12  console.log(rect);
13
14  var Square = function (width) {
15      Rectangle.call(this, width, width);
16  };
17
18  Square.prototype = new Rectangle();
19
20  var sq = new Square(5);
21  console.log(sq)
22
23  var rect2 = new sq.constructor(2, 3);
24  console.log(rect2);
25
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe ex.js
Rectangle { width: 3, height: 4 }
Rectangle { width: 5, height: 5 }
Rectangle { width: 2, height: 3 }
```

클래스가 구체적인 데이터를 지니지 않게 하는 방법1

그림 7-8 Rectangle → Square 상속 관계 구현 (1) – 도식

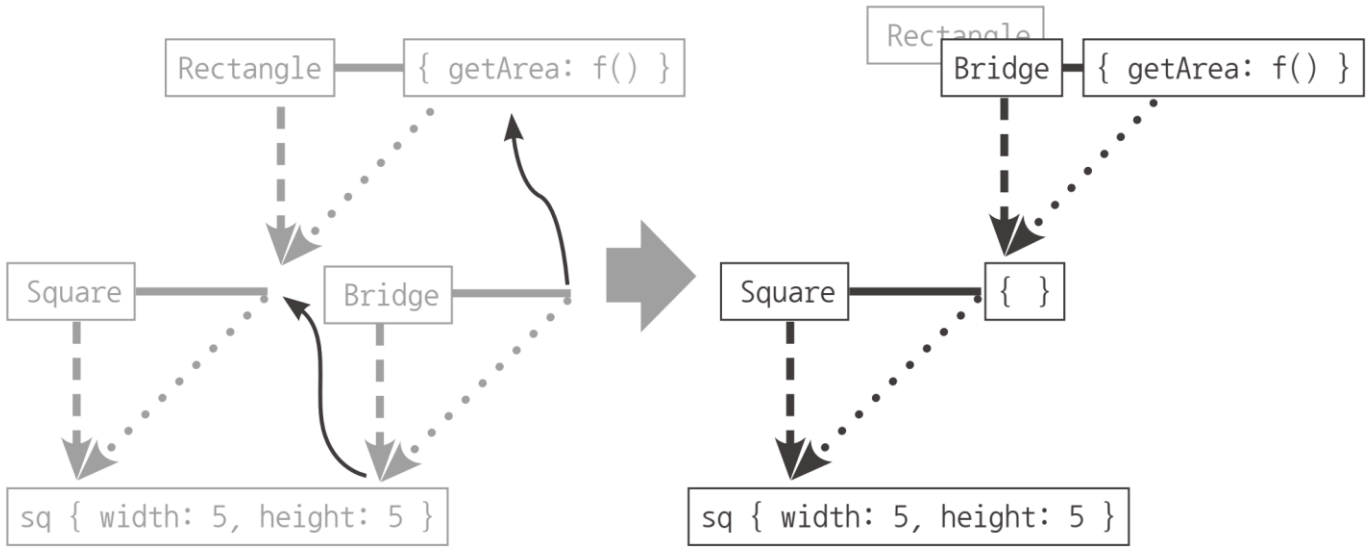



```
25
26    delete Square.prototype.width;
27    delete Square.prototype.height;
28    Object.freeze(Squere.prototype)
29
```

```
1
2   var extendClass = function (SuperClass, SubClass) {
3       SubClass.prototype = new SuperClass();
4       for (var prop in SubClass.prototype) {
5           if (SubClass.prototype.hasOwnProperty(prop)) {
6               delete SubClass.prototype[prop];
7           }
8       }
9       Object.freeze(SubClass.prototype);
10      return SubClass;
11  };
12
13  var Rectangle = function (width, height) {
14      this.width = width;
15      this.height = height;
16  };
17
18  Rectangle.prototype.getArea = function () {
19      return this.width * this.height;
20  };
21
22  var Square = extendClass(Rectangle, function (width) {
23      Rectangle.call(this, width, width);
24  });
25
26  var sq = new Square(5);
27  console.log(sq.getArea());
28
```

클래스가 구체적인 데이터를 지니지 않게 하는 방법2

그림 7-9 클래스 상속 및 추상화 방법(2) – 빈 함수를 활용

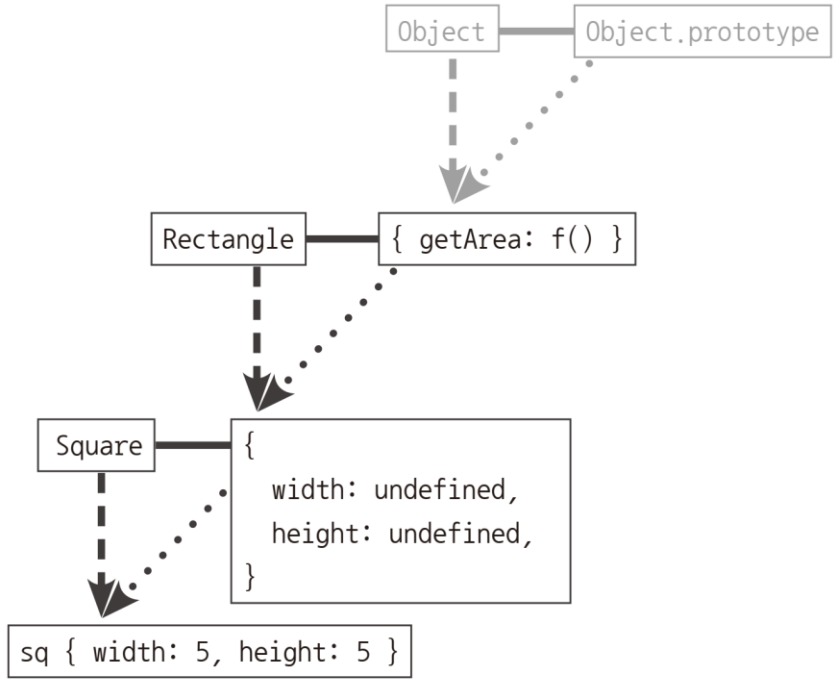



```
var Bridge = function() {};
Bridge.prototype = Rectangle.prototype;
Square.prototype = new Bridge();
Object.freeze(Square.prototype);
```

```
1
2   var extendClass = (function () {
3       var Bridge = function () { };
4       return function (SuperClass, SubClass) {
5           Bridge.prototype = SuperClass.prototype;
6           SubClass.prototype = new Bridge();
7           Object.freeze(SubClass.prototype);
8           return SubClass;
9       };
10  })();
11
12  var Rectangle = function (width, height) {
13      this.width = width;
14      this.height = height;
15  };
16
17  Rectangle.prototype.getArea = function () {
18      return this.width * this.height;
19  };
20
21  var Square = extendClass(Rectangle, function (width) {
22      Rectangle.call(this, width, width);
23  });
24
25  var sq = new Square(5);
26  console.log(sq.getArea());
27
```

클래스가 구체적인 데이터를 지니지 않게 하는 방법3

그림 7-8 Rectangle → Square 상속 관계 구현 (1) – 도식




```
1
2   var Rectangle = function (width, height) {
3       this.width = width;
4       this.height = height;
5   };
6
7   Rectangle.prototype.getArea = function () {
8       return this.width * this.height;
9   };
10
11  var Square = function (width) {
12      Rectangle.call(this, width, width);
13  };
14
15  Square.prototype = Object.create(Rectangle.prototype);
16  Object.freeze(Square.prototype);
17
18  var sq = new Square(5);
19  console.log(sq.getArea());
20
21
```

ES6의 클래스

```
1
2   var ES5 = function (name) {
3       this.name = name;
4   };
5
6   ES5.staticMethod = function () {
7       return this.name + ' staticMethod';
8   };
9
10  ES5.prototype.method = function () {
11      return this.name + ' method';
12  };
13
14  var es5Instance = new ES5('es5');
15  console.log(ES5.staticMethod());
16  console.log(es5Instance.method());
17
```

```
1
2   var ES6 = class {
3       constructor(name) {
4           this.name = name;
5       }
6       static staticMethod() {
7           return this.name + ' staticMethod';
8       }
9       method() {
10          return this.name + ' method';
11      }
12  };
13  var es6Instance = new ES6('es6');
14  console.log(ES6.staticMethod());
15  console.log(es6Instance.method());
16
```

ES6의 클래스 상속

```
1
2   var Rectangle = class {
3       constructor(width, height) {
4           this.width = width;
5           this.height = height;
6       }
7       getArea() {
8           return this.width * this.height;
9       }
10  };
11
12  var Square = class extends Rectangle {
13      constructor(width) {
14          super(width, width);
15      }
16      getArea() {
17          console.log('size is :', super.getArea());
18      }
19  };
20
```

Thank You!